

RabbitMQ

Java Backend Interview — Short Notes

1. Core Concepts & Architecture

- **RabbitMQ**: open-source message broker implementing AMQP (Advanced Message Queuing Protocol). Push-based.
- **Producer**: application that publishes messages to an exchange.
- **Exchange**: receives messages from producer and routes to queues based on type and routing key.
- **Queue**: buffer that stores messages until consumed. FIFO by default.
- **Consumer**: application that subscribes to a queue and processes messages.
- **Binding**: link between exchange and queue. Defines routing rule (routing key or pattern).
- **Virtual Host (vhost)**: logical separation of resources — exchanges, queues, bindings. Like DB schema.
- **Channel**: lightweight connection inside a TCP connection. Multiplex multiple operations over one connection.

1.1 Message Flow

- Producer → Exchange → Binding (routing key) → Queue → Consumer
- Messages are NOT sent directly to queues. They go through exchange which routes based on type and routing key.

1.2 Exchange Types

Exchange Type	Routing Logic	Use Case	Example Binding
Direct	Route to queue whose binding key exactly matches message routing key.	Task distribution — route by job type. Error routing by severity.	binding key='payment' → payment queue. key='email' → email queue.
Topic	Route to queues whose binding pattern matches (wildcards: * = one word, # = zero or more).	Multi-criteria routing — log level + service. Event filtering.	Binding '*.error' matches 'payment.error'. '#' matches all.
Fanout	Broadcast to ALL bound queues. Ignores routing key.	Notifications, live updates, cache invalidation — all consumers get message.	order.placed → fanout → email queue + SMS queue + analytics queue.
Headers	Route based on message headers (key-value pairs). More flexible than routing keys.	Complex multi-attribute routing where topic patterns are insufficient.	x-match=all: headers {format:pdf, type:report} → report queue.
Default (nameless)	Routes to queue whose name equals routing key. Every queue auto-bound.	Simple direct queue publishing.	Publish with routing key='my-queue' → goes to queue named 'my-queue'.

2. Message Properties & Durability

2.1 Message Persistence

- **Transient messages**: stored in memory only. Lost if RabbitMQ restarts.
 - **Persistent messages**: delivery_mode=2. Written to disk. Survive broker restart.
 - **Durable queue**: queue definition survives restart. Must ALSO have persistent messages for full durability.
 - **Durable exchange**: exchange definition survives restart.
- Durable queue alone is NOT enough — messages must also be persistent (delivery_mode=2). Both required.

2.2 Acknowledgment Modes

Mode	Description	Pros	Cons	Use When
Auto ACK	Message removed from queue immediately upon delivery to consumer.	Simple, high throughput.	Message lost if consumer crashes before processing.	Non-critical, idempotent, at-most-once acceptable.
Manual ACK	Consumer explicitly calls <code>basicAck()</code> after processing.	At-least-once delivery — no message loss.	Unacked messages redelivered if consumer crashes — duplicates possible.	Critical business messages. Default recommendation.
Negative ACK (NACK)	Consumer rejects message: <code>requeue=true</code> puts back in queue; <code>requeue=false</code> drops or DLX.	Control over failed processing.	<code>Requeue=true</code> can cause infinite loop on poison messages.	Transient failures with retry. Fatal errors: <code>nack + requeue=false</code> → DLX.

3. Advanced Features

3.1 Dead Letter Exchange (DLX)

- Messages routed to DLX when: rejected (`nack/reject` with `requeue=false`), TTL expired, queue max-length exceeded.
- Use DLX for: error handling, delayed retry, poison message isolation, audit trail.

```
// Queue with DLX configuration
Map<String, Object> args = new HashMap<>();
args.put("x-dead-letter-exchange", "dlx.orders");
args.put("x-dead-letter-routing-key", "failed");
args.put("x-message-ttl", 30000); // 30s TTL
args.put("x-max-length", 10000); // max queue depth
channel.queueDeclare("orders", true, false, false, args);
```

3.2 Retry with TTL + DLX (Delayed Retry Pattern)

- Pattern: message fails → `nack` → routed to wait queue (TTL) → TTL expires → routed back to main queue.

```
// Wait queue: expires after 5s, routes back to main exchange
args.put("x-message-ttl", 5000);
args.put("x-dead-letter-exchange", "main.exchange");
args.put("x-dead-letter-routing-key", "orders");
channel.queueDeclare("wait.orders", true, false, false, args);
```

3.3 Prefetch (QoS) — Fair Dispatch

- **prefetchCount=1**: broker doesn't send new message to consumer until previous one is acked.
- Prevents fast consumers from getting all messages while slow consumers are idle.
- Higher prefetch = better throughput. Lower = fairer distribution and better memory control.

```
channel.basicQos(1); // prefetch 1 message at a time per consumer
```

3.4 Publisher Confirms

- Lightweight alternative to transactions. Broker acks each published message when safely persisted.
- Channel in confirm mode: `channel.confirmSelect()`. `basicPublish()` then `waitForConfirms()`.
- Without confirms: fire-and-forget. Message may be lost if broker restarts before writing to disk.

```
channel.confirmSelect();
```

```
channel.basicPublish(exchange, routingKey, props, body);
if (!channel.waitForConfirms(5000)) throw new Exception("Publish not confirmed");
```

4. Spring AMQP & Spring Boot Integration

4.1 Configuration

```
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guest
    password: guest
    virtual-host: /
    listener:
      simple:
        acknowledge-mode: manual
        prefetch: 10
        concurrency: 3
        max-concurrency: 10

// Declare exchange, queue, binding as beans
@Configuration public class RabbitConfig {
  @Bean TopicExchange ordersExchange() {
    return new TopicExchange("orders.exchange", true, false);
  }
  @Bean Queue ordersQueue() {
    return QueueBuilder.durable("orders.queue")
      .withArgument("x-dead-letter-exchange", "dlx.exchange")
      .build();
  }
  @Bean Binding binding(Queue q, TopicExchange ex) {
    return BindingBuilder.bind(q).to(ex).with("order.#");
  }
}
```

4.2 Producer & Consumer

```
// Producer
@Service public class OrderEventPublisher {
  @Autowired RabbitTemplate rabbitTemplate;

  public void publishOrderCreated(Order order) {
    rabbitTemplate.convertAndSend(
      "orders.exchange",
      "order.created", // routing key
      order,
      msg -> {
        msg.getMessageProperties().setDeliveryMode(PERSISTENT);
        msg.getMessageProperties().setExpiration("300000");
        return msg;
      });
  }
}

// Consumer with manual ack
```

```

@Component public class OrderConsumer {
    @RabbitListener(queues = "orders.queue", ackMode = "MANUAL")
    public void handle(Order order, Channel channel,
        @Header(DELIVERY_TAG) long tag) throws IOException {
        try {
            processOrder(order);
            channel.basicAck(tag, false); // ack = processed OK
        } catch (RetryableException e) {
            channel.basicNack(tag, false, true); // requeue = retry
        } catch (FatalException e) {
            channel.basicNack(tag, false, false); // → DLX
        }
    }
}

```

5. RabbitMQ vs Kafka — Decision Guide

Dimension	RabbitMQ	Kafka
Model	Push-based. Broker pushes to consumer.	Pull-based. Consumer polls broker.
Message retention	Deleted after consumption (acked).	Retained for configured period. Replay possible.
Throughput	Thousands/sec. Lower than Kafka.	Millions/sec via partitioning.
Ordering	Per queue (FIFO).	Per partition (by key).
Routing	Powerful: direct/topic/fanout/headers exchanges.	Topic-based only. Consumer group handles competing.
Consumer groups	Competing consumers — message to one consumer.	Multiple independent groups — all get message.
Replay	Not supported — message gone after ack.	Core feature — replay from any offset.
Protocol	AMQP (open standard). MQTT, STOMP also.	Proprietary Kafka protocol.
Best for	Task queues, RPC-style, complex routing, short-lived messages. Traditional MQ use cases.	Event streaming, audit log, event sourcing, high-volume, multiple independent consumers.
Choose when	Need flexible routing, priority queues, per-message TTL, DLX retry patterns, AMQP compliance.	Need replay, high throughput, multiple consumers reading same events, long retention.

6. Interview Questions

Core Concepts

Q1. What is AMQP? How does RabbitMQ use it?

→ AMQP (Advanced Message Queuing Protocol): open wire-level protocol for messaging. Defines: exchanges, queues, bindings, message acknowledgments, publisher confirms. RabbitMQ implements AMQP 0-9-1. Consumers connect via TCP, open channels, declare resources, publish/consume messages using AMQP protocol frames.

Q2. Explain the role of Exchange in RabbitMQ.

→ Exchange receives messages from producers and routes them to queues based on exchange type and routing key. Producers never publish directly to queues. 4 types: Direct (exact key match), Topic (wildcard pattern), Fanout (broadcast all), Headers (header attributes). Binding links exchange to queue with optional routing key.

Q3. What happens to unacknowledged messages if consumer crashes?

→ With manual ack: unacknowledged messages return to the queue when consumer connection drops. They are redelivered (redelivered=true flag set) to another consumer. Ensures at-least-once delivery. With auto-ack: message is lost on crash — at-most-once semantics.

Advanced

Q4. How do you implement retry with backoff in RabbitMQ?

→ Pattern: (1) Consumer fails → `basicNack(requeue=false)` → message routes to DLX. (2) DLX routes to wait queue with `x-message-ttl` (e.g., 5s). (3) Wait queue TTL expires → DLX routes back to main queue. (4) Repeat with longer wait queues (5s, 30s, 5min). Include `retry-count` header to stop after N attempts and send to permanent DLQ.

Q5. What is prefetch count? Why set it?

→ `prefetchCount` limits how many unacked messages broker sends to a consumer at once. `prefetch=1`: fairest distribution — consumer only gets new message after acking previous. Higher prefetch = higher throughput (fewer round trips) but less fair distribution. Set prefetch based on processing time: fast processing → high prefetch. Slow → low prefetch.

Q6. How do you guarantee exactly-once delivery with RabbitMQ?

→ RabbitMQ guarantees at-least-once with manual ack + durable queue + persistent messages. True exactly-once requires idempotent consumers. Pattern: include unique message ID in payload. Consumer checks idempotency key in DB before processing. If already processed: ack and skip. Outbox pattern ensures message published if DB transaction succeeds.

Q7. RabbitMQ vs Kafka — when to pick RabbitMQ?

→ Choose RabbitMQ when: need complex routing (topic patterns, header-based), per-message TTL/priority, traditional task queue semantics (one consumer per message), short-lived messages (no replay needed), AMQP protocol requirement, RPC-style request-reply pattern. Choose Kafka when: need replay, multiple independent consumers, high throughput, event sourcing.